

Technical Documentation: RAG Customer Support Assistant

1. Introduction

Retrieval-Augmented Generation (RAG) is a technique that grounds Large Language Models (LLMs) in external, factual knowledge bases. Instead of relying solely on an LLM's pre-trained knowledge, RAG retrieves relevant documents and passes them to the LLM to generate an informed answer.

Why it is needed: LLMs tend to hallucinate when asked about proprietary or domain-specific information (like a company's internal policies). RAG mitigates this by providing concrete facts.

Use Case Overview: A Customer Support Assistant that answers client queries using a company's PDF manual. If the bot is unsure or the user requires complex help, the system escalates the ticket to a human agent.

2. System Architecture Explanation

The system follows a bipartite architecture: Data Preparation and Inference.

1. Data Preparation: PDFs are loaded via [PyPDFLoader], split into 1000-character chunks to maintain semantic coherence, embedded into numerical vectors using an embedding model, and persisted in a ChromaDB instance.
2. Inference (LangGraph): The core logic is orchestrated by LangGraph. A user's query enters the StateGraph. A routing node determines the query's intent (Standard vs. Escalation). If standard, the retriever fetches relevant chunks from ChromaDB, and the LLM synthesizes an answer. If escalation is required, the graph transitions to the Human-in-the-Loop (HITL) node.

3. Design Decisions

- Chunk Size Choice: 1000 characters with a 200-character overlap. This size is large enough to contain full paragraphs (complete thoughts) but small enough to fit multiple chunks into an LLM's context window without exceeding token limits.
- Embedding Strategy: OpenAI Embeddings or HuggingFace embeddings are used for their high accuracy in capturing semantic meaning, essential for accurate retrieval.
- Retrieval Approach: Top-K similarity search (k=3 or 4) using cosine similarity. It balances context richness and prompt size.
- Prompt Design Logic: The LLM prompt explicitly instructs the model: "You are a support assistant. Answer the question using ONLY the provided context. If you do not know the answer, say 'I do not know' and trigger an escalation."

4. Workflow Explanation

LangGraph Usage: LangGraph manages the cyclical and conditional flow of the agent.

- Node Responsibilities:
 - [router_node]: Decides where the query goes.
 - [rag_node]: Performs the actual retrieval and generation.
 - [hitl_node]: Handles escalation logic.
- State Transitions: The application maintains a [TypedDict] state containing the user query, retrieved

context, and the final response. State transitions happen strictly along defined edges based on the router's output.

5. Conditional Logic

- Intent Detection: Uses a lightweight LLM call or regex keyword matching to detect the user's intent.
- Routing Decisions:
- IF intent == "escalate" OR query == "complex" -> Route to [hitl_node]
- ELSE -> Route to [rag_node]

6. HITL Implementation

- Role of Human Intervention: The HITL mechanism ensures that the AI does not confidently provide wrong answers to sensitive customer issues. It serves as a safety net.
- Benefits: High customer satisfaction, reduced legal/business risk, continuous learning (human answers can be fed back into the DB).
- Limitations: Increases response time for escalated queries; requires maintaining a human workforce.

7. Challenges & Trade-offs

- Retrieval Accuracy vs. Speed: Using highly complex embeddings (e.g., cross-encoders) improves accuracy but increases latency. We opted for fast bi-encoder embeddings (ChromaDB default).
- Chunk Size vs. Context Quality: Smaller chunks yield precise matches but might miss surrounding context. Overlap mitigates this trade-off.
- Cost vs. Performance: Using GPT-4o provides the best reasoning for routing and answering but is costly. A trade-off is using a smaller model like GPT-4o-mini or Llama-3 for standard queries.

8. Testing Strategy

- Testing Approach:
- 1. Component testing: Verify document chunking and ChromaDB retrieval independently.
- 2. Integration testing: Ensure the LangGraph transitions state correctly based on mock intents.
- 3. End-to-End testing: Run sample queries through the entire pipeline.
- Sample Queries:
- Standard: "What is your return policy?"
- Escalation: "I want to speak to a manager about my broken product."
- Out of Domain: "What is the capital of France?" (Should be gracefully rejected).

9. Future Enhancements

- Multi-document support: Extend the loader to handle entire directories of PDFs, DOCX, and HTML files.
- Feedback loop: Allow users to rate answers (thumbs up/down) to fine-tune the retrieval and generation phases.
- Memory Integration: Implement LangGraph's checkpointing to maintain conversation history for multi-turn chats.

- Deployment: Containerize the application using Docker and deploy as a microservice on AWS or GCP.